



UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA

Trabalho sobre Métodos de Classificação

ALUNO:

FELIPE NASCIMENTO DE ANGELIS 130131

ANÁLISE DE ALGORITMOS E A.EM GRAFOS

MARINGÁ-PR

2025

Os códigos utilizados neste trabalho estão disponíveis em um arquivo compactado no formato **.zip**. Para acessar os scripts, basta descompactar o arquivo. O objetivo deste estudo é comparar e analisar a performance de diferentes métodos de ordenação, incluindo o InsertionSort, BubbleSort, SelectionSort, ShellSort, QuickSort, HeapSort, MergeSort e RadixSort, para diferentes cenários de dados (ordenados de forma crescente, decrescente e aleatória). O desempenho de cada algoritmo será avaliado em termos de tempo de execução e eficiência, considerando as respectivas complexidades temporais e comportamentos nos diversos casos analisados.

Análise dos Resultados - Método de Inserção Direta

O método de **Inserção Direta (Insertion Sort)** é um algoritmo simples e intuitivo, mas não é eficiente para grandes conjuntos de dados. Ele funciona bem para conjuntos pequenos ou parcialmente ordenados. Vamos considerar os tempos observados para os diferentes cenários:

1. Dados em Ordem Crescente:

- Nesse caso, o algoritmo realiza apenas comparações sem movimentar elementos, pois a lista já está ordenada.
- O tempo de execução será o menor entre os três cenários, apresentando comportamento **linear** $O(n)$

2. Dados em Ordem Decrescente:

- Este é o pior caso para o algoritmo. Cada elemento será comparado com todos os elementos anteriores e movido até a posição correta.
- O número de comparações e movimentações será máximo, resultando em uma complexidade **quadrática** $O(n^2)$
- O tempo observado será significativamente maior do que o tempo para dados crescentes.

3. Dados em Ordem Aleatória:

- Neste caso, o tempo de execução dependerá do grau de desordem dos dados.
- Em média, o desempenho também será **quadrático** $O(n^2)$, embora o tempo observado possa variar dependendo do tamanho do vetor e da distribuição dos números.

Conclusão:

O **Insertion Sort** apresenta desempenho satisfatório para listas pequenas ou quase ordenadas, mas seu uso em grandes conjuntos de dados é limitado devido à complexidade $O(n^2)$ no pior e no caso médio.

Análise dos Resultados - Método da Bolha (BubbleSort)

O **BubbleSort** é um dos algoritmos de ordenação mais simples, mas também um dos menos eficientes para grandes conjuntos de dados. Ele funciona ao comparar elementos adjacentes e trocá-los se estiverem fora de ordem, repetindo esse processo várias vezes até que a lista esteja ordenada. Aqui está a análise dos tempos esperados para diferentes cenários:

1. Dados em Ordem Crescente:

- Nesse caso, o BubbleSort ainda percorrerá o vetor inteiro, mas nenhuma troca será realizada.
- Mesmo sem trocas, ele verifica todos os pares de elementos em cada iteração, resultando em $O(n^2)$ comparações no pior caso, mas o tempo observado será menor em comparação aos dados em ordem decrescente ou aleatória.

2. Dados em Ordem Decrescente:

- Este é o pior caso para o algoritmo. Cada elemento terá que ser movido até a posição correta no final da lista.
- Isso requer o máximo número de comparações e trocas, resultando em uma complexidade $O(n^2)$.
- O tempo de execução será significativamente maior que para os dados crescentes.

3. Dados em Ordem Aleatória:

- Neste caso, o desempenho dependerá do grau de desordem dos dados, mas a complexidade ainda será $O(n^2)$.
- O tempo observado estará geralmente entre o tempo dos dados em ordem crescente (melhor caso) e decrescente (pior caso).

Conclusão:

O **BubbleSort** não é adequado para ordenar grandes conjuntos de dados devido à sua baixa eficiência $O(n^2)$, tanto no caso médio quanto no pior caso. Ele pode ser útil para fins educacionais ou para pequenos conjuntos de dados onde a simplicidade do algoritmo é mais relevante que o desempenho.

Análise dos Resultados - Método de Seleção Direta (Selection Sort)

O **Selection Sort** é um algoritmo de ordenação simples que funciona selecionando o menor elemento de uma lista não ordenada e colocando-o na posição correta. Esse processo se repete para cada elemento até que a lista esteja ordenada. Abaixo, analisamos o desempenho para diferentes cenários:

1. Dados em Ordem Crescente:

- Mesmo com os dados já ordenados, o algoritmo ainda verifica todos os elementos restantes para encontrar o menor.
- Não há ganhos em termos de desempenho, e o número de comparações permanece $O(n^2)$.
- Apesar disso, o tempo de execução será menor do que no pior caso, já que as trocas serão reduzidas.

2. Dados em Ordem Decrescente:

- Este é o pior caso para o Selection Sort. O algoritmo fará $O(n^2)$ comparações e também várias trocas para reorganizar os elementos.
- Como cada elemento está fora de posição, o tempo de execução será mais alto do que no caso crescente.

3. Dados em Ordem Aleatória:

- No caso médio, o número de comparações ainda é $O(n^2)$, independentemente da ordem dos dados.
- O tempo observado dependerá da distribuição dos números, mas será geralmente intermediário entre os cenários de ordem crescente e decrescente.

Conclusão:

O **Selection Sort** é simples de implementar e pode ser útil para conjuntos de dados pequenos ou quando o consumo de memória precisa ser minimizado, já que é um algoritmo **in-place** (não requer espaço adicional significativo). No entanto, sua eficiência é baixa para listas grandes devido à complexidade $O(n^2)$.

Análise dos Resultados - Método ShellSort

O **Shell Sort** é uma generalização do **Insertion Sort** que melhora o desempenho ao permitir comparações e trocas de elementos distantes. Ele reduz gradualmente o intervalo de comparação (**gap**) até chegar a 1, onde o algoritmo se comporta como o Insertion Sort. Vamos analisar o desempenho para diferentes cenários:

1. Dados em Ordem Crescente:

- Para dados já ordenados, o Shell Sort realiza poucas movimentações, já que os elementos estão em suas posições corretas.
- O número de comparações e movimentações será mínimo em relação aos outros cenários.
- O desempenho é consideravelmente melhor que o Insertion Sort, pois o uso de gaps permite menos iterações iniciais.

2. Dados em Ordem Decrescente:

- Este é o pior caso para o Shell Sort, já que os elementos estão na ordem inversa.

- No entanto, devido ao uso de gaps, o Shell Sort ainda supera o Insertion Sort e o Bubble Sort em eficiência, mesmo neste cenário.
- A complexidade pode variar dependendo da sequência de gaps utilizada, mas geralmente está entre $O(n^{\{3/2\}})$ e $O(n^2)$.

3. Dados em Ordem Aleatória:

- Para dados aleatórios, o desempenho dependerá do grau de desordem, mas o Shell Sort geralmente apresenta melhor eficiência do que os algoritmos simples como o Bubble Sort e o Selection Sort.
- O uso de gaps reduz drasticamente o número de comparações e movimentações necessárias.

Conclusão:

O **Shell Sort** é um algoritmo intermediário em termos de eficiência. Embora sua complexidade dependa da escolha da sequência de gaps, ele é geralmente mais rápido que os algoritmos simples $O(n^2)$ e menos eficiente do que os algoritmos avançados como **QuickSort** ou **MergeSort**.

Pontos Fortes:

- Simples de implementar e mais eficiente que o Bubble Sort ou o Insertion Sort.
- Bom para conjuntos de dados de tamanho moderado.

Pontos Fracos:

- O desempenho é sensível à escolha da sequência de gaps.
- Menos eficiente que algoritmos de complexidade $O(n \log n)$ para grandes conjuntos de dados.

Análise dos Resultados - Método QuickSort

O **QuickSort** é um dos algoritmos de ordenação mais eficientes e amplamente utilizados. Ele é baseado no conceito de dividir e conquistar, escolhendo um pivô para particionar a lista em sublistas menores e ordenando essas sublistas recursivamente. Aqui está a análise de desempenho para os diferentes cenários:

1. Dados em Ordem Crescente:

- No caso fornecido, onde o pivô é escolhido como o elemento do meio, os dados em ordem crescente não afetam negativamente o desempenho.
- A complexidade será $O(n \log n)$, pois o algoritmo particiona bem os dados e os divide em partes aproximadamente iguais.

- O tempo de execução será rápido, refletindo a eficiência do QuickSort nesse cenário.

2. Dados em Ordem Decrescente:

- Quando os dados estão em ordem decrescente, o algoritmo ainda funcionará bem, pois o pivô escolhido como elemento central garante uma boa divisão dos dados.
- Diferente de implementações que escolhem o primeiro ou o último elemento como pivô, esta abordagem evita o pior caso $O(n^2)$ neste cenário.
- O desempenho será semelhante ao caso crescente, mantendo a complexidade $O(n \log n)$.

3. Dados em Ordem Aleatória:

- Este é o cenário mais típico para o QuickSort. O desempenho será $O(n \log n)$ na maioria das vezes, pois a escolha do pivô como o elemento central lida bem com a aleatoriedade.
- O algoritmo demonstra sua melhor eficiência nesse tipo de dado, sendo geralmente mais rápido do que os métodos simples e até mesmo comparável a outros algoritmos avançados, como MergeSort.

Conclusão:

O **QuickSort** é extremamente eficiente para a maioria dos conjuntos de dados devido à sua complexidade média $O(n \log n)$ e baixa sobrecarga. Com a escolha de um bom pivô, ele evita o pior caso e mantém o desempenho estável.

Pontos Fortes:

- Muito rápido para conjuntos de dados grandes e moderadamente grandes.
- Bom desempenho em dados aleatórios e parcialmente ordenados.
- Usa espaço adicional limitado, já que é um algoritmo in-place (em algumas implementações iterativas).

Pontos Fracos:

- Pode atingir o pior caso $O(n^2)$ dependendo da escolha do pivô (o que não é o caso deste código).

Análise dos Resultados - Método HeapSort

O **HeapSort** é um algoritmo de ordenação baseado na estrutura de dados "heap". Ele utiliza um heap máximo para ordenar os elementos, garantindo que o maior elemento seja removido e colocado em sua posição final a cada iteração. Este

método possui desempenho previsível e é eficiente para conjuntos de dados grandes.

Resultados para Diferentes Tipos de Dados

1. Dados em Ordem Crescente:

- O **HeapSort** é independente da ordem inicial dos dados, pois sempre reconstrói o heap a partir dos elementos fornecidos.
- A complexidade é $O(n \log n)$, e o tempo de execução será constante e previsível.
- Apesar disso, o processo de reconstruir o heap pode ser ligeiramente menos eficiente quando os dados já estão ordenados.

2. Dados em Ordem Decrescente:

- Semelhante ao caso crescente, o **HeapSort** lida bem com dados em ordem decrescente.
- A reconstrução do heap máximo garante que a complexidade permaneça $O(n \log n)$.
- A ordem inicial dos dados não causa impacto significativo no tempo de execução.

3. Dados em Ordem Aleatória:

- O desempenho do **HeapSort** é ideal para dados aleatórios, mantendo sua complexidade $O(n \log n)$.
- Este é o cenário típico em que o algoritmo demonstra sua eficiência e previsibilidade.

Conclusão:

O **HeapSort** é uma escolha robusta para ordenação de grandes volumes de dados, especialmente quando a previsibilidade do desempenho é crucial. Embora não seja tão rápido quanto o **QuickSort** na maioria dos casos, ele oferece um tempo de execução estável e consistente.

Análise dos Resultados - Método MergeSort

O algoritmo **MergeSort** possui complexidade $O(n \log n)$ no pior, melhor e caso médio, o que significa que sua performance não é afetada pela ordem dos elementos de entrada. No entanto, ao medir o tempo de execução em diferentes cenários, podemos observar:

Resultados esperados

1. Vetor ordenado crescente:

- O **MergeSort** deve apresentar uma performance constante e estável em todos os tamanhos de vetor. Isso ocorre porque o algoritmo recursivo sempre divide o vetor em duas metades, independente da ordem dos elementos.
- O tempo de execução será diretamente proporcional ao tamanho do vetor **$O(n \log n)$** , sem diferenças significativas entre os cenários.

2. Vetor ordenado decrescente:

- Mesmo com os dados em ordem decrescente, o **MergeSort** ainda opera de forma eficiente. Não importa a ordem inicial dos dados, pois o algoritmo sempre divide recursivamente e combina as metades de maneira eficiente.
- O tempo de execução também será **$O(n \log n)$** , e a performance será praticamente a mesma do caso anterior.

3. Vetor aleatório:

- No caso de dados aleatórios, a performance do **MergeSort** será igualmente **$O(n \log n)$** . Em comparação com os cenários anteriores, pode haver variações mínimas no tempo devido à distribuição aleatória dos números, mas a complexidade de tempo permanece constante.

Conclusão:

À medida que o tamanho do vetor aumenta, o tempo de execução também deve aumentar proporcionalmente, de acordo com a complexidade **$O(n \log n)$** . Isso significa que:

- Para vetores de 1000 elementos, o tempo de execução será razoavelmente rápido.
- Para vetores de 5000 ou 10000 elementos, a diferença de tempo se tornará mais perceptível, mas o crescimento será controlado pela natureza do algoritmo **MergeSort**.

Análise dos Resultados - Método RadixSort

O código mede o tempo de execução do **RadixSort** em três cenários diferentes: vetores ordenados de forma crescente, decrescente e aleatória. Ele avalia também três tamanhos de vetores: 1000, 5000 e 10000.

Resultados esperados

1. Vetor ordenado crescente:

- O **RadixSort** não é sensível à ordem dos dados, portanto, a performance deve ser a mesma independentemente da ordem inicial

dos elementos. O tempo de execução depende mais do número de dígitos e do tamanho do vetor.

- Para vetores em ordem crescente, o **RadixSort** deve apresentar um tempo de execução semelhante ao dos outros cenários, com a complexidade $O(d * (n + k))$, onde d é o número de dígitos, n é o tamanho do vetor e k é a base do sistema numérico (neste caso, 10).

2. Vetor ordenado decrescente:

- Como o **RadixSort** processa os dígitos independentemente da ordem dos números, o tempo de execução para dados decrescentes será basicamente o mesmo que para dados crescentes. A lógica de ordenação se baseia nos dígitos e não em comparações entre os elementos.
- A performance será igualmente eficiente, com o tempo de execução escalando de acordo com a complexidade mencionada.

3. Vetor aleatório:

- Para dados aleatórios, o **RadixSort** funciona da mesma forma que nos outros casos. A diferença no tempo de execução será mínima, uma vez que o algoritmo ordena com base em cada dígito, sem se importar com a ordem original dos elementos.
- O tempo de execução pode variar ligeiramente devido à distribuição aleatória dos números, mas o **RadixSort** mantém sua eficiência com $O(d * (n + k))$.

Conclusão:

O aumento do tamanho do vetor terá um impacto no tempo de execução, mas o **RadixSort** ainda se manterá eficiente comparado a algoritmos de ordenação como **QuickSort** e **MergeSort** em cenários de grandes volumes de dados. Isso ocorre porque a complexidade do **RadixSort** não depende de comparações diretas entre os elementos, mas sim de operações baseadas nos dígitos.

- Para vetores pequenos (como 1000 elementos), o tempo de execução será rápido.
 - Para vetores maiores (5000 e 10000 elementos), o tempo de execução será mais perceptível, mas a eficiência do algoritmo se mantém, com uma complexidade proporcional ao número de dígitos.
 -
-

- **Inserção Direta**

```
Tamanho do vetor: 1000
Tempo para crescente: 0.000000 segundos
Tempo para decrescente: 0.062864 segundos
Tempo para aleatório: 0.018461 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.000000 segundos
Tempo para decrescente: 1.348564 segundos
Tempo para aleatório: 0.591705 segundos

Tamanho do vetor: 10000
Tempo para crescente: 0.001026 segundos
Tempo para decrescente: 4.407069 segundos
Tempo para aleatório: 2.031839 segundos
```

- **BubbleSort - Bolha**

```
Tamanho do vetor: 1000
Tempo para crescente: 0.027439 segundos
Tempo para decrescente: 0.102243 segundos
Tempo para aleatório: 0.118201 segundos

Tamanho do vetor: 5000
Tempo para crescente: 1.124139 segundos
Tempo para decrescente: 1.698573 segundos
Tempo para aleatório: 1.165053 segundos

Tamanho do vetor: 10000
Tempo para crescente: 3.071955 segundos
Tempo para decrescente: 6.611012 segundos
Tempo para aleatório: 5.253077 segundos
```

- **Seleção Direta**

```
Tamanho do vetor: 1000
Tempo para crescente: 0.034905 segundos
Tempo para decrescente: 0.052377 segundos
Tempo para aleatório: 0.032913 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.646412 segundos
Tempo para decrescente: 0.636015 segundos
Tempo para aleatório: 0.489569 segundos

Tamanho do vetor: 10000
Tempo para crescente: 1.947775 segundos
Tempo para decrescente: 2.091743 segundos
Tempo para aleatório: 2.051064 segundos
```

- **ShellSort**

```
Tamanho do vetor: 1000
Tempo para crescente: 0.005001 segundos
Tempo para decrescente: 0.003974 segundos
Tempo para aleatório: 0.005984 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.010973 segundos
Tempo para decrescente: 0.018463 segundos
Tempo para aleatório: 0.022937 segundos

Tamanho do vetor: 10000
Tempo para crescente: 0.011969 segundos
Tempo para decrescente: 0.028951 segundos
Tempo para aleatório: 0.030489 segundos
```

- QuickSort

```
Tamanho do vetor: 1000
Tempo para crescente: 0.001994 segundos
Tempo para decrescente: 0.001996 segundos
Tempo para aleatório: 0.003989 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.012966 segundos
Tempo para decrescente: 0.011972 segundos
Tempo para aleatório: 0.014477 segundos

Tamanho do vetor: 10000
Tempo para crescente: 0.024931 segundos
Tempo para decrescente: 0.030918 segundos
Tempo para aleatório: 0.029438 segundos
```

- HeapSort

```
Tamanho do vetor: 1000
Tempo para crescente: 0.001995 segundos
Tempo para decrescente: 0.002992 segundos
Tempo para aleatório: 0.005986 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.028923 segundos
Tempo para decrescente: 0.026445 segundos
Tempo para aleatório: 0.026929 segundos

Tamanho do vetor: 10000
Tempo para crescente: 0.048866 segundos
Tempo para decrescente: 0.044396 segundos
Tempo para aleatório: 0.069331 segundos
```

- MergeSort

```
Tamanho do vetor: 1000
Tempo para crescente: 0.001992 segundos
Tempo para decrescente: 0.001997 segundos
Tempo para aleatório: 0.001995 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.018950 segundos
Tempo para decrescente: 0.017951 segundos
Tempo para aleatório: 0.024457 segundos

Tamanho do vetor: 10000
Tempo para crescente: 0.035904 segundos
Tempo para decrescente: 0.035928 segundos
Tempo para aleatório: 0.066337 segundos
```

- RadixSort

```
Tamanho do vetor: 1000
Tempo para crescente: 0.001995 segundos
Tempo para decrescente: 0.000996 segundos
Tempo para aleatório: 0.002988 segundos

Tamanho do vetor: 5000
Tempo para crescente: 0.014959 segundos
Tempo para decrescente: 0.016954 segundos
Tempo para aleatório: 0.018466 segundos

Tamanho do vetor: 10000
Tempo para crescente: 0.033909 segundos
Tempo para decrescente: 0.038896 segundos
Tempo para aleatório: 0.034422 segundos
```
